

Project Report: Smart MCQ Solver Challenge

Name: Vatsal Gupta

Roll Number: 23F3002118

Deep Learning & Generative AI Project Report

1. Abstract / Executive Summary

Project Overview: This document provides a comprehensive report detailing the work performed on the Smart MCQ Solver Challenge Kaggle competition.

Problem Statement: The objective required designing an artificial intelligence system capable of solving complex multiple-choice questions containing a prompt and five options labeled A through E.

Methodologies: The project explored four progressive modeling strategies, including a TF-IDF baseline, a custom BiLSTM with Attention, a pretrained BERT transformer, and a weighted ensemble.

Key Results: Performance was evaluated using Mean Average Precision at 3 (MAP@3), with the weighted ensemble and BiLSTM models achieving top scores of 0.9988.

Conclusions: Integrating traditional statistical features with deep sequence models and transformer architectures yields exceptional answer-ranking capabilities.

2. Introduction

Problem Statement: The competition challenges participants to build AI systems that not only identify correct answers for multiple-choice questions but also rank alternative options effectively based on confidence and contextual understanding. The dataset comprises questions featuring prompts and five choices (A-E).

Project Objective: Specific goals included achieving high predictive accuracy under the MAP@3 evaluation metric, mastering tools like PyTorch, Hugging Face, and Weights & Biases, and experimenting with diverse GenAI architectures.

Report Structure: The subsequent sections outline dataset preprocessing, tokenization strategies, model experimentation, performance analysis, and project conclusions.

3. Dataset & Preprocessing

Dataset Description: The competition data consists of train.csv with 2,000 rows containing prompts, options A–E, and correct answer labels, alongside test.csv containing 500 challenge questions without answers. A sample_submission.csv template defines the required output format.

Exploratory Data Analysis (EDA): Initial analysis inspected answer distributions, confirming a balanced mix of correct answers across options A through E, and analyzed prompt word lengths using histogram visualizations.

Data Preprocessing: Data cleaning involved converting entries to strings, applying regular expressions (`re.sub(r"\s+", " ", text)`) to normalize whitespace, and setting up label encoding mapping answer letters ('A'-'E') to integer indices (0-4).

Train/Validation Split: The training dataset was split into an 80% training set (1,600 samples) and a 20% validation set (400 samples) using stratified sampling (`stratify=train["answer"]`) to maintain balanced class distributions.

4. Tokenization Strategy

Primary Tokenizer Selection: The primary tokenizer utilized was bert-base-uncased via Hugging Face's AutoTokenizer, associated with the BERT transformer architecture.

Justification: Subword tokenization effectively handles out-of-vocabulary terms and captures linguistic sub-structures essential for contextual comprehension.

Implementation Details: Tokenization pipelines were efficiently implemented using Hugging Face libraries with appropriate padding and truncation strategies for batch processing.

5. Modeling & Experimentation

5.1. Model 1: Fine-Tuned Transformer Model (Pretrained BERT)

Architecture: Selected bert-base-uncased implemented via AutoModelForMultipleChoice from Hugging Face.

Salient Points: Leverages multi-head self-attention mechanisms and deep encoder stacks to model complex contextual relationships between prompts and answer choices.

Fine-Tuning Strategy: Trained using the AdamW optimizer with a learning rate of 2×10^{-5} across epochs, tracking loss and metric progression via Weights & Biases (Wandb).

5.2. Model 2: Custom Deep Learning Model (BiLSTM + Attention)

Architecture: Developed a custom neural network from scratch comprising an embedding layer, a bidirectional LSTM (batch_first=True, bidirectional=True), an attention linear layer, and a final classification layer.

Salient Points: Designed to sequentially process token embeddings while dynamically weighing the importance of specific hidden states via the attention mechanism.

5.3. Model 3: Statistical Baseline Model (TF-IDF + Logistic Regression)

Architecture: Implemented using scikit-learn's TfidfVectorizer to extract word n-gram frequency features paired with LogisticRegression for multi-class probability estimation.

Salient Points: Serves as a lightweight, robust statistical baseline providing rapid inference and solid predictive performance before deploying deep neural networks.

5.4. Weighted Ensemble

Architecture: Combined validation probabilities from all three models using optimized weights: 15% TF-IDF + Logistic Regression, 25% BiLSTM + Attention, and 60% Pretrained BERT.

Salient Points: Leverages the distinct inductive biases of statistical, sequential, and transformer models to maximize overall prediction stability and accuracy'.

6. Performance & Comparative Analysis

Evaluation Metrics: Models were evaluated using Mean Average Precision at 3 (MAP@3) as the primary competition metric, alongside Accuracy and Macro F1-score.

Training Performance: Training runs, loss curves, and validation metrics were logged and monitored online using Weights & Biases (wandb).

Comparative Report:

TF-IDF + Logistic Regression: Accuracy 0.9750, Macro F1 0.9740, MAP@3 0.9858

BiLSTM + Attention: Accuracy 0.9975, Macro F1 0.9974, MAP@3 0.9988

Pretrained BERT: Accuracy 0.9800, Macro F1 0.9792, MAP@3 0.9896

Weighted Ensemble: Accuracy 0.9975, Macro F1 0.9974, MAP@3 0.9988

Kaggle Performance: Final test set predictions were generated, structured according to sample_submission.csv, and successfully exported for competition evaluation.

7. Conclusion & Future Work

Key Learnings: The project provided practical insights into data preprocessing, building custom recurrent neural networks, fine-tuning large pre-trained transformers, and executing ensemble strategies within a complete machine learning lifecycle.

Challenges Faced: Overcame technical obstacles including Kaggle environment dependency resolution and experiment tracking configuration with Weights & Biases.

Areas for Improvement: Future work could explore advanced transformer variations, larger pre-trained language models, retrieval-augmented generation (RAG), and broader hyperparameter optimization techniques.

8. References

PyTorch (torch, torch.nn) Deep Learning Framework

Hugging Face Transformers (AutoTokenizer, AutoModelForMultipleChoice)

Scikit-Learn (TfidfVectorizer, LogisticRegression, train_test_split, metrics)

Pandas and NumPy for Data Processing and Numerical Operations

Weights & Biases (wandb) for Experiment Tracking